

ChronoGuard-EHR: Availability Aware Data Management for Point in Time Clinical Analytics

Yan Li¹, Jialu Zhao², Hongzhe Zhang¹, Zhe Xue¹, Meiyu Liang¹, Guanhua Ye¹, and Yawen Li^{1*}

¹ Beijing University of Posts and Telecommunications, Beijing, China

² State Information Center, Beijing, China

Email: {liyanly,zhanghongzhe,xuezh,zhe,xuezh,meiyu1210,g.ye,warmly0716}@bupt.edu.cn
zhaojialu@sic.gov.cn

Abstract. Longitudinal electronic health record analytics often constructs features from event occurrence time: a laboratory result, bedside vital sign, or intake/output record is treated as available once its clinical timestamp precedes the prediction time. Retrospective electronic health records do not satisfy this assumption. A clinical fact may occur before it enters an operational system or becomes visible to clinicians or prediction services; consequently, occurrence time only queries may include values that were unavailable at the decision point, creating hidden point in time errors. This paper presents ChronoGuard-EHR, an availability aware data management system for point in time clinical analytics. ChronoGuard-EHR uses executable time contracts to declare source fields as occurrence time, availability time, and encounter boundary roles; applies hardening policies that prevent inconsistent timestamps from causing premature visibility while preserving raw anomalies as audit evidence; and compiles contracts into safe extraction predicates for batch feature extraction and bounded streaming replay. We evaluate the system on MIMIC-IV and eICU workloads. The results show that independent availability time fields exhibit nontrivial visibility delays, contract hardening removes execution level premature visibility, and availability aware extraction changes event inclusion and feature construction. Independent relational query baselines over raw source views further validate that ChronoGuard-EHR agrees with manually written safe predicates and exposes records admitted by occurrence time only filtering. Bounded replay experiments show that the same semantics preserve output correctness while revealing measurable service costs caused by late arriving information. These results demonstrate that availability time should be treated as a first class data management semantic for reliable longitudinal EHR analytics.

Keywords: Electronic health records · Point in time correctness · Temporal data management · Clinical analytics

* Corresponding author.

1 Introduction

Large longitudinal electronic health record datasets are becoming core infrastructure for clinical analytics and clinical machine learning. Critical care resources such as MIMIC-IV and the eICU Collaborative Research Database expose large scale timestamped measurements, orders, interventions, and outcomes, supporting studies of mortality risk, length of stay estimation, deterioration warning, operational monitoring, and resource allocation [1,2,3]. These datasets are retrospective databases. After a hospitalization or ICU stay has ended, analysts can query many facts that were not visible to clinicians, monitoring systems, or prediction models at the historical decision time. This error usually does not appear as an explicit future event. Instead, it is hidden among multiple plausible but semantically different timestamps within the same clinical table. Prior clinical machine learning leakage studies show that leakage cannot be avoided by the simple rule “do not use future data”; whether a variable leaks depends on the prediction time, prediction viewpoint, label definition, and deployment time availability [4]. Empirical analyses of MIMIC-IV style workflows also illustrate that post hoc information, such as same encounter diagnosis codes, can create severe same stay leakage and produce deceptively high model performance.

Existing EHR feature extraction commonly relies on occurrence time filtering: if a record’s clinical timestamp is earlier than the prediction time, the system includes it in the feature window. This rule is not sufficient for point in time correctness. A laboratory specimen may be collected inside the prediction window but written back to the system only after the prediction time. A nursing note may describe an earlier bedside observation but be entered or revised after the decision. Administrative and diagnosis codes may be finalized only after discharge. In these cases, the event itself has occurred, but the value bearing database fact was not visible at the time. Models trained or evaluated on such records observe the completed retrospective database state rather than the information state available at deployment.

This paper formulates the mismatch as a query semantics problem. For point in time prediction and longitudinal clinical analytics, a database system must determine not only whether a fact occurred before the decision time, but also whether that fact was available at that time according to the source table’s temporal semantics. We call the latter timestamp role *availability time*. Availability time is related to valid time and transaction time in temporal databases, and to point in time joins in feature stores [5,6,7]. The EHR setting introduces additional complications: a single table may contain sampling time, charting time, storage time, revision time, and coding time after discharge; timestamp roles vary across table families; some tables lack explicit availability fields; and raw timestamps may be inverted, backfilled, or entered late. A generic as of join can answer which version existed before a time, but it cannot decide which clinical timestamp should serve as the eligibility boundary for prediction.

We present ChronoGuard-EHR, an availability aware query execution layer for longitudinal EHR analytics. ChronoGuard-EHR declares table level temporal

semantics as executable contracts. Each contract specifies occurrence time roles, availability time roles, encounter boundaries, missingness policies, and uncertainty handling rules. The system first validates contract coverage and audits timestamp anomalies. It then constructs effective availability times through hardening policies, preventing raw timestamp inversions from making records prematurely visible while preserving raw anomalies as audit evidence. Finally, the safe extractor compiles contracts into point in time predicates for batch feature extraction and bounded streaming replay. The process outputs aggregate audit reports, difference summaries, and run manifests; patient level facts, feature matrices, and prediction details remain inside the controlled environment.

We evaluate ChronoGuard-EHR on two public EHR datasets, MIMIC-IV and eICU. The experiments show that availability time is not decorative metadata but a core semantic that determines whether a clinical query is point in time correct. In real EHR tables, occurrence time and availability time exhibit substantial delays and anomalies. Occurrence time only filtering admits records invisible at the prediction time. Contract based execution eliminates execution level premature visibility while preserving raw anomalies for audit. Safe extraction changes event inclusion, feature values, and missingness patterns. In streaming settings, visibility semantics further expose real information availability delay and service deadline pressure. Downstream model diagnostics show that the effect of visibility constraints on model metrics, calibration, and train/deployment mismatch is task dependent; therefore, visibility should be an explicit audit dimension in clinical model evaluation rather than an implicit assumption inside feature engineering scripts.

The contributions are:

- We formalize point in time correctness in longitudinal EHR queries as a safety constraint over occurrence time, availability time, effective availability time, and table level time contracts, thereby separating “the event occurred” from “the fact was visible.”
- We introduce ChronoGuard-EHR, which compiles source table time semantics into executable point in time predicates and supports contract validation, timestamp anomaly audit, availability hardening, safe feature extraction, aggregate difference reporting, and bounded streaming replay.
- We quantify differences between occurrence time and availability time on MIMIC-IV and eICU longitudinal workloads and evaluate how availability aware extraction changes event inclusion, feature construction, missingness, and streaming service cost relative to occurrence time only filtering.
- We analyze how visibility constraints affect downstream model metrics, calibration, and train/deployment mismatch, showing that the effect is task dependent and should be audited explicitly during model evaluation.

2 Related Work

Temporal data management and point in time features. Temporal data management has long studied the relationship among facts, database state, and query time. Valid time describes when a fact holds in the real world, transaction time describes when a fact is recorded by the database, and bitemporal models preserve both dimensions to support historical tracking, audit, and temporal queries [5,6]. More recently, machine learning data management and feature store systems have treated training data construction as a data systems problem. Historical retrieval and as of joins recreate the feature state associated with a given entity and timestamp, thereby avoiding training data that reads future records [7]. ML systems work also links feature construction to technical debt and automated data quality checks [8,9].

These systems provide an important foundation for point in time correctness, but they often assume that each feature view already has a trustworthy timestamp. Longitudinal EHR data has a more upstream difficulty: the same clinical table may contain sampling time, charting time, storage time, revision time, and coding completion time, and the semantics of these fields differ across table families. This concern is consistent with EHR data quality and reuse studies, which emphasize that completeness, provenance, and local workflow context shape secondary analyses [10,11,12]. The problem is therefore not only how to execute an as of join, but which timestamp should serve as the visibility boundary. ChronoGuard-EHR makes this prerequisite explicit through table level time contracts and handles missing availability, timestamp inversions, and uncertain temporal semantics during execution.

Data provenance, audit, and streaming time. Data provenance research studies how query outputs are produced from input data and supports debugging, explanation, and trustworthy computation [13,14]. Traditional provenance methods explain why an output appears, where an output value originates, and how input tuples propagate to query results. ChronoGuard-EHR adopts this audit perspective, but its witness is more specific than general lineage: it is time role aware leakage evidence. A witness identifies the relevant source table, occurrence time role, availability time role, missingness or inversion policy, and query location that allowed an invisible record to enter a result.

Clinical prediction benchmarks and leakage. ICU prediction benchmarks and broader EHR modeling standardize cohorts, tasks, and metrics for mortality, length of stay, and decompensation style endpoints [3,15]. ChronoGuard is complementary to these benchmarks: it does not propose a new model family, but checks whether the feature relation supplied to any model obeys the information boundary at the prediction time. This positioning is also consistent with recent leakage analyses, which emphasize that deployment time availability must be specified rather than assumed [4]. Broader health AI and data mining studies further illustrate how feature construction, missingness, representation learning, and task context appear across lesion segmentation, sleep apnea detection, missing data imputation,

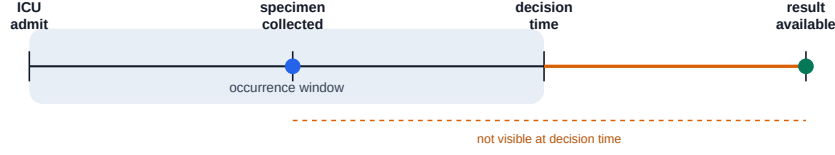


Fig. 1. Occurrence time and availability time define different eligibility boundaries. A result can occur within a prediction window while becoming visible only after the decision time; availability aware extraction masks the value rather than treating it as observed.

sequential recommendation, questionnaire representation, ESG forecasting, and machine unlearning [16,17,18,19,20,21,22].

Streaming data systems also distinguish event occurrence time from system processing time, using watermarks and late data mechanisms to balance correctness, latency, and cost over out of order streams [23,24,25,26]. ChronoGuard-EHR shares the basic observation that event occurrence time and system visible time are not the same. However, this paper does not build a general purpose streaming engine. Instead, it executes clinical visibility contracts over retrospective EHR data and bounded replay. The streaming evaluation uses safe full recomputation as the correctness baseline and quantifies how availability delay affects service deadlines, thereby turning the question of when a clinical fact becomes visible into an auditable system constraint.

3 Problem Definition

We model a clinical fact as an event

$$e = (u, x, c, v, t_o, t_a, s),$$

where u is a patient or stay entity, x is an encounter context, c is a concept, v is a value, t_o is the occurrence time, t_a is the raw availability time, and s is the source relation. A decision time workload evaluates features at a time d relative to an encounter anchor such as ICU admission.

The occurrence only predicate is

$$P_{\text{occ}}(e, d) \equiv t_o(e) \leq d.$$

The availability aware predicate is

$$P_{\text{safe}}(e, d) \equiv t_o(e) \leq d \wedge t_a^*(e) \leq d,$$

where t_a^* is the effective availability time constructed from the raw timestamp and the source contract. A feature $f(d)$ is point in time correct under the contract semantics if every event in its lineage satisfies $P_{\text{safe}}(e, d)$. Thus an event may occur before d and still be excluded because its value was not visible at d .

For a concept family C_j , let

$$E_j^{\text{safe}}(d) = \{e \mid c(e) \in C_j, P_{\text{safe}}(e, d)\}.$$

A numeric feature is computed as $g_j(E_j^{\text{safe}}(d))$, where g_j may be count, min, max, mean, last value, total, or rate. If the set is empty, the value feature is masked and auxiliary features such as visible count, missingness, recency, and pending status may be emitted only when those signals are themselves observable.

Definition 1 (Time contract). *A time contract is a tuple $\kappa_s = (r_o, r_a, r_b, \pi_m, \pi_i)$ for a source relation s , where r_o identifies the occurrence role, r_a identifies the availability role, r_b identifies encounter boundary roles, π_m handles missing availability, and π_i handles timestamp inversions. A query is safe only with respect to a declared contract.*

The contract does not assert that the source timestamps are clinically perfect. It defines what the execution engine is allowed to treat as visible. Raw inconsistencies remain auditable.

4 Availability Semantics and Contracts

A contract assigns every included table to a visibility policy. For example, a laboratory table may use collection time as occurrence and result storage time as availability, while a vital sign table may use chart time and entry time when both are present. If no independent availability field exists, the contract either excludes the table from strict mode or marks an availability equals occurrence assumption with lower confidence.

A frequent anomaly is an apparent inversion, $t_a < t_o$. This can occur through back entry, correction, batch ingestion, or field semantics that do not encode true result visibility. ChronoGuard-EHR does not delete these records. For strict execution it constructs

$$t_a^*(e) = \max(t_o(e), t_a(e)),$$

while retaining the raw inversion $t_a < t_o$ as audit metadata. This hardening policy prevents execution level premature visibility and preserves evidence that the source timestamps are inconsistent.

Proposition 1 (Audit preserving hardening). *For any event e with an occurrence time and raw availability time, $t_a^*(e) = \max(t_o(e), t_a(e))$ satisfies $t_a^*(e) \geq t_o(e)$. If $t_a(e) < t_o(e)$, that raw inversion remains observable in the audit relation.*

Proof (Proof sketch). The max operator enforces the lower bound t_o for execution. The raw fields are not overwritten; the inversion predicate is evaluated over the raw timestamps and stored as aggregate audit evidence.

Algorithm 1 Contract validation and hardening**Require:** Source relation R_s , contract κ_s **Ensure:** Hardened relation H_s , audit summary A_s

- 1: Bind occurrence expression r_o and availability expression r_a
- 2: Mark rows with missing availability according to π_m
- 3: Count raw inversions $r_a(e) < r_o(e)$
- 4: **for all** events $e \in R_s$ **do**
- 5: **if** availability is executable **then**
- 6: $t_a^*(e) \leftarrow \max(r_o(e), r_a(e))$
- 7: **else**
- 8: apply the contract's strict exclusion or low confidence proxy policy
- 9: **end if**
- 10: **end for**
- 11: Emit H_s with t_a^* and emit audit summary A_s

Algorithm 2 Availability aware feature extraction**Require:** Hardened events H , decision time d , feature specification Φ

- 1: **for all** feature definitions $\phi_j \in \Phi$ **do**
- 2: Select $E_j \leftarrow \{e \in H \mid c(e) \in C_j, t_o(e) \leq d, t_a^*(e) \leq d\}$
- 3: Compute declared aggregates over E_j
- 4: Emit missingness, count, and recency indicators from visible events only
- 5: **end for**

Theorem 1 (Safe extraction soundness). *If a feature extractor emits only aggregates over events satisfying $P_{\text{safe}}(e, d)$, then every emitted feature is point in time correct under the declared contracts.*

Proof (Proof sketch). The feature lineage consists only of admitted events. For every admitted event, the safe predicate requires both occurrence and effective availability no later than d . No value unavailable at d can contribute to the aggregate.

5 ChronoGuard System

Fig. 2 shows the system. ChronoGuard-EHR separates visibility semantics from feature definitions. A contract catalog stores timestamp roles and policies. A validator checks schema bindings, contract coverage, missing availability fields, and raw inversions. The hardener constructs effective availability timestamps and audit summaries. The extractor compiles feature specifications into predicates over P_{safe} . Reports and manifests record aggregate outputs, runtime metadata, and artifact digests.

For validation, we also execute independent DuckDB baselines on bounded raw source views. These baselines are used to check semantic behavior, not to claim optimizer superiority: occurrence only SQL exposes visibility violations, while contract driven predicates remove them by enforcing availability time eligibility.

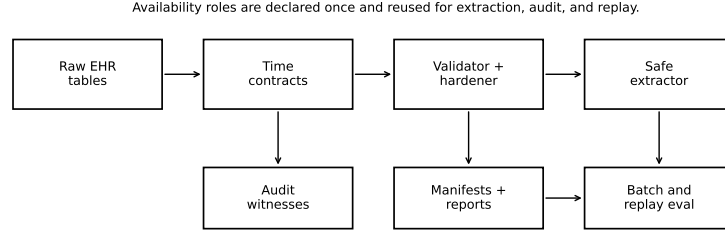


Fig. 2. ChronoGuard compiles table level time contracts into safe predicates, hardens inconsistent timestamps without erasing raw audit evidence, and records aggregate artifacts for batch extraction and bounded replay.

6 Experiments

The evaluation is organized by analytical function rather than by research question labels. It first characterizes availability timing in source tables, then validates contract enforcement and query audit, quantifies extraction impact, evaluates bounded streaming cost, and audits downstream prediction impact.

6.1 Datasets

The main experiments use two public deidentified EHR datasets: MIMIC-IV and the eICU Collaborative Research Database. MIMIC-IV provides structured tables for hospital admissions, ICU stays, vital signs, laboratory tests, intake/output, orders, diagnoses, and outcomes, and is the primary evaluation dataset. eICU provides multicenter ICU data and is used to test whether time contracts apply across different EHR structures.

All experiments output only aggregate statistics, table level reports, feature differences, model metrics, and runtime summaries. We do not export patient level facts, stay level feature matrices, label details, prediction probabilities, or raw clinical rows.

6.2 Workloads and Baselines

The batch workload is built around early ICU analysis at 360, 720, and 1440 minutes after ICU admission. Features include last values, means, extrema, counts, missingness indicators, and urine output totals for vital signs, laboratory tests, and intake/output records.

The streaming workload simulates rolling prediction. It repeatedly computes safe features at multiple decision times and compares safe full recomputation with a streaming reuse path.

Dataset	Table family	P50	P95	Mean	Max	Neg. lag rate
MIMIC-IV	Charted vitals	18	187	55.1	3.16M	0.084
	Laboratory results	73	459	142.7	1.00M	0.000
	Output events	17	149	38.0	527K	0.141
eICU	Laboratory results	40	134	65.2	50.0M	0.050
	Nurse charting	0	0	21.4	7.36M	0.009
	Intake/output	0	0	54.9	481K	0.033

Table 1. Representative availability profiles in ICU EHR tables. Contract validated occurrence to availability pairs show nontrivial central delays in laboratory and charted event tables, while maximum delays and negative lag rates expose timestamp outliers that require hardening before point in time extraction.

The model evaluation workload holds the cohort, labels, prediction times, feature set, model class, and data split fixed while changing only the feature filtering rule. This design isolates the downstream effect of visibility constraints.

We compare two execution modes.

Occurrence only includes records solely by requiring the event occurrence time to be no later than the prediction time.

ChronoGuard-EHR uses table level time contracts to identify the appropriate occurrence and availability roles, constructs effective availability time, and executes the resulting safe predicates.

6.3 Metrics

Temporal semantics metrics. We report availability missingness rate, median and 95th percentile visibility delay, negative delay rate, and long tail delay rate.

Point in time correctness metrics. We report violation counts, violation rates, raw timestamp inversion counts, effective inversion counts after hardening, contract generated safe query results, and witness localization accuracy.

Event and feature impact metrics. We report event exclusion fraction, feature changed fraction, mean absolute change, and missingness change.

Streaming system metrics. We report output match rate, compute latency, availability delay, end to end latency, service deadline violation rate, and peak memory.

Model evaluation metrics. We report AUROC, AUPRC, Brier score, expected calibration error, calibration slope, and metric changes under train/test mismatch.

Dataset	Source table	Raw inv.	Eff. inv.	Excl. frac.	Feat. chg.
MIMIC-IV					
	chart events	36.5M	0	0.096	0.497
	lab events	n/a	0	0.086	0.113
	output events	757.8K	0	0.101	0.107
eICU					
	intake/output	392.6K	0	0.118	0.107
	lab	1.97M	0	0.084	0.056
	nurse charting	1.29M	0	0.048	0.120
	vital periodic	0	0	0.000	0.120

Table 2. Hardening eliminates effective premature visibility while retaining raw inversion evidence for audit. Fractions are aggregate maxima across evaluated horizons.

6.4 Visibility Time in EHR Tables

We perform contract validation and table level aggregate analysis on source tables that expose both an occurrence time role and an independent availability time role. For each table family, we compute the occurrence to availability delay distribution, including the median, 95th percentile, mean, and maximum delay, and we report the raw negative lag rate. The median and 95th percentile summarize common visibility delay, the mean and maximum reveal long tail behavior, and the negative lag rate captures raw timestamp anomalies in which availability time precedes occurrence time. Together, these metrics assess whether table level temporal semantics are sufficient for safe point in time extraction.

Table 1 shows that occurrence time and availability time are not interchangeable in core EHR tables. In MIMIC-IV, laboratory results have a median visibility delay of 73 minutes and a 95th percentile delay of 459 minutes; bedside charted vital signs have a median delay of 18 minutes and a 95th percentile delay of 187 minutes. In eICU, laboratory results have a median delay of 40 minutes and a 95th percentile delay of 134 minutes. Some documentation style tables have medians and 95th percentiles near or equal to zero, but still exhibit nonzero means, large maximum delays, and negative lag rates. Thus, the common path and tail anomalies must be modeled separately rather than treating these sources as zero delay tables.

These results support the central data management premise of the paper: a timestamped clinical fact may have occurred while still being invisible to a query at the same decision time. Visibility problems arise not only as stable central delays in laboratory and bedside event tables, but also as the coexistence of missingness, long tail delays, and inverted timestamp roles. Reliable point in time EHR analytics therefore cannot rely on a single occurrence time predicate; it must explicitly declare, validate, and execute table specific visibility semantics.

Query class	Risk	Rewrite action	Witness
Selection	occurrence only filter	add $t_a^* \leq t_d$	table, role
Window agg.	invisible rows in window	filter before aggregation	inserted predicate
As of join	unbound timestamp	bind to availability role	join boundary
Anti join	hindsight exclusion	require visible evidence	relation role
Admin. code	finalized code leakage	reject or flag	finalization risk

Table 3. ChronoGuard rewrites unsafe query idioms by inserting contract derived visibility predicates and emitting compact audit witnesses. Ambiguous cases are rejected or flagged rather than silently certified.

6.5 Contract Enforcement and Auditability

The contract enforcement evaluation tests whether executable time contracts can enforce point in time eligibility while preserving evidence about timestamp anomalies. We apply contract validation and hardening policies to the same source families and summarize raw timestamp inversions, effective inversions after hardening, event exclusion fractions, and feature changed fractions. To test whether the system goes beyond handwritten filtering scripts, we also evaluate representative unsafe query idioms through contract compilation and query rewriting. The reported metrics include raw inversion counts, effective inversion counts, maximum event exclusion fraction, and maximum feature changed fraction. For query rewriting, we record the unsafe query class, the associated visibility risk, the generated rewrite action, and the audit witness emitted by the contract compiler. The witness is aggregate level evidence that identifies the source table, time roles, and unsafe query pattern without exposing patient level records.

Table 2 shows that raw timestamp inversions can be common in real EHR tables, but after contract hardening all included table families have zero effective inversions. The largest raw inversion count in MIMIC-IV occurs in chart events; eICU laboratory and nurse charting tables also expose large raw anomaly counts. Table 3 further shows that selection, window aggregation, as of join, anti join, and administrative code access can be expressed as contract driven query rewriting problems rather than ad hoc manual filters. Table 4 exposes the tradeoff among hardening policies: the default clamp policy prevents execution level premature visibility while preserving the raw anomaly signal for audit.

These results show that ChronoGuard-EHR is not merely a reminder that users can write safer SQL manually. Its system role is to compile table level visibility semantics into executable predicates, produce compact audit evidence for unsafe query patterns, and separate execution eligibility from anomaly

Policy	Retained	Cov. loss	Eff. inv.	Audit	Use
Raw availability	0.962	0.038	122.9M	yes	diagnostic
Clamp to occurrence	0.962	0.038	0	yes	default
Exclude inverted rows	0.484	0.516	0	yes	sensitivity
Occurrence proxy	1.000	0.000	0	no	low conf.
Strict missing exclusion	0.962	0.038	0	yes	sensitivity
Assume equal to occurrence	1.000	0.000	0	yes	assumption

Table 4. Hardening policies trade measurable coverage against execution safety and auditability. The default clamp policy retains the same aggregate event coverage as raw availability while reducing effective premature visibility inversions from 122.9M to zero and preserving the raw anomaly audit signal.

preservation. In this way, availability time becomes a query processing contract rather than an informal data cleaning convention.

6.6 Extraction Impact

For each prediction time, we compare occurrence only extraction with availability aware extraction under the same cohort, source tables, and feature definitions. The only difference is the eligibility rule: the occurrence only extractor requires event occurrence time no later than the prediction time, whereas the availability aware extractor also requires effective availability time no later than the prediction time. At the event level, we report the fraction of occurrence eligible records excluded by the availability predicate. At the feature level, we report the fraction of stays whose feature values change under safe extraction, organized by dataset, feature, and prediction time. All statistics are aggregate; no stay level feature vectors are exported.

The largest early horizon event exclusions concentrate in table families with natural documentation or result availability delay. At the 6 h prediction time, eICU intake/output excludes 11.8% of occurrence eligible events, MIMIC-IV output events exclude 10.1%, MIMIC-IV charted events exclude 9.6%, and MIMIC-IV laboratory events exclude 8.6%. Fig. 3 further shows that feature changes are not uniform: MIMIC-IV vital sign count features change for roughly one third to one half of stays across prediction times, whereas several laboratory and intake/output summary features have smaller but still measurable changes.

This experiment connects timestamp semantics to the input relation consumed by downstream analytics. Availability aware execution changes the relation before any model training, risk score, or aggregate statistic is computed. The most affected families are precisely those whose documentation or result availability is naturally delayed. This also explains why feature level point in

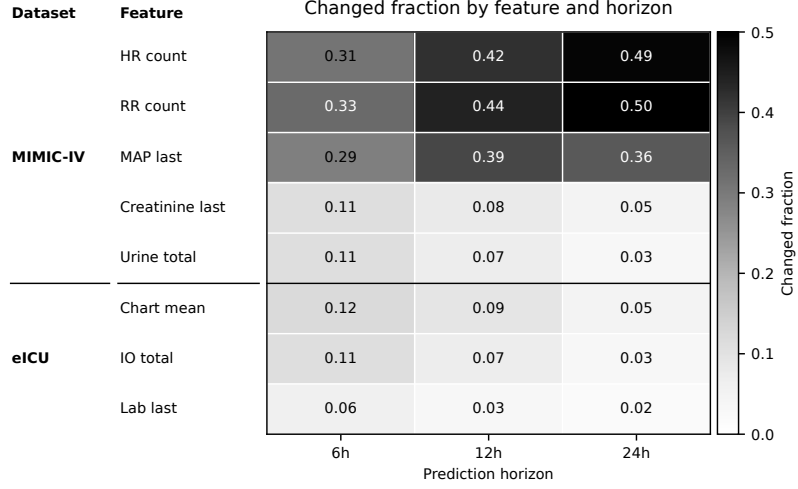


Fig. 3. Visibility sensitive feature changes are shown by dataset, feature, and prediction horizon. The heatmap reports aggregate changed fractions for representative MIMIC-IV and eICU rows; it is not a clinical causal analysis.

time changes can be substantial even when downstream model metrics are not always markedly different. Fig. 3 visualizes the changed fractions for visibility sensitive features across datasets, feature names, and prediction times.

6.7 Streaming Correctness and Cost

We compare safe full recomputation with an incremental update prototype in bounded event replay and check whether the two paths produce the same outputs under the same availability eligibility rules. We measure output match rate, compute latency, information availability delay, late arrival correction, service deadline violation rate, and the latency difference between the incremental prototype and full recomputation. We distinguish two latency sources: compute latency comes from system processing and feature updates, whereas availability delay comes from the time at which a clinical fact enters the visible information state. The former is an execution cost; the latter is an information availability constraint.

The bounded replay results show that the incremental prototype reproduces the outputs of safe full recomputation, but it is not faster in the current implementation. Fig. 4 shows that compute latency and information availability delay are distinct quantities: even when feature computation is fast, a clinical fact may still be unavailable and cause a service deadline violation. Service pressure therefore does not always originate from computational bottlenecks; it can also arise from availability semantics.

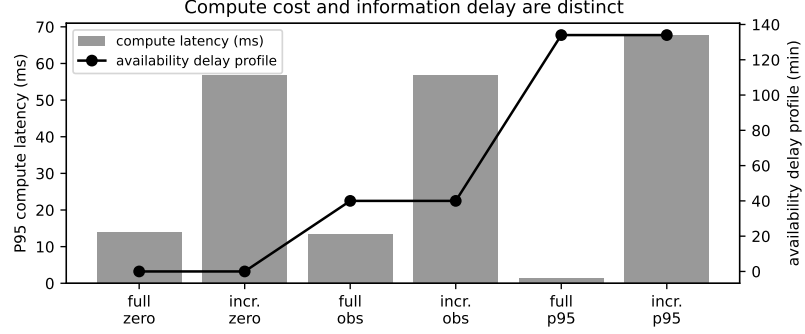


Fig. 4. Availability aware replay separates compute latency from information availability delay. The bounded incremental prototype preserves outputs relative to full recomputation but is not claimed to provide speedup.

These results extend the ChronoGuard-EHR system claim from batch extraction to streaming analysis. Availability aware streaming execution must reason jointly about processing cost and information arrival. The correct conclusion is a correctness and cost characterization: the prototype preserves bounded replay output semantics, while incremental optimization remains separate engineering work.

6.8 Downstream Prediction Impact

We compare two matched training and testing semantics under the same tasks, data splits, model classes, and feature definitions: availability aware training with availability aware testing, and occurrence only training with occurrence only testing. The tasks are mortality, long stay, and ICU readmission; the models are logistic regression and gradient boosted trees. We report aggregate AUROC and Brier score values for each task, prediction time, and model. AUROC measures discrimination, and Brier score measures probabilistic prediction error. The experiment exports only aggregate metrics; prediction probabilities, labels, and patient level feature matrices are not exported.

Fig. 5 summarizes representative results. For these broad outcome tasks, availability aware safe/safe evaluation and occurrence only unsafe/unsafe evaluation are often close, with differences that are small and task dependent. Repeated permutation diagnostics do not support a main text claim that occurrence only evaluation is systematically optimistic. In other words, feature level visibility changes do not necessarily translate into large model metric shifts for all outcome tasks.

This finding is important for a systems paper. The value of ChronoGuard-EHR should not be framed as universally improving model accuracy or proving that all traditional models are overoptimistic. A more accurate interpretation is that ChronoGuard-EHR makes training and testing information availability

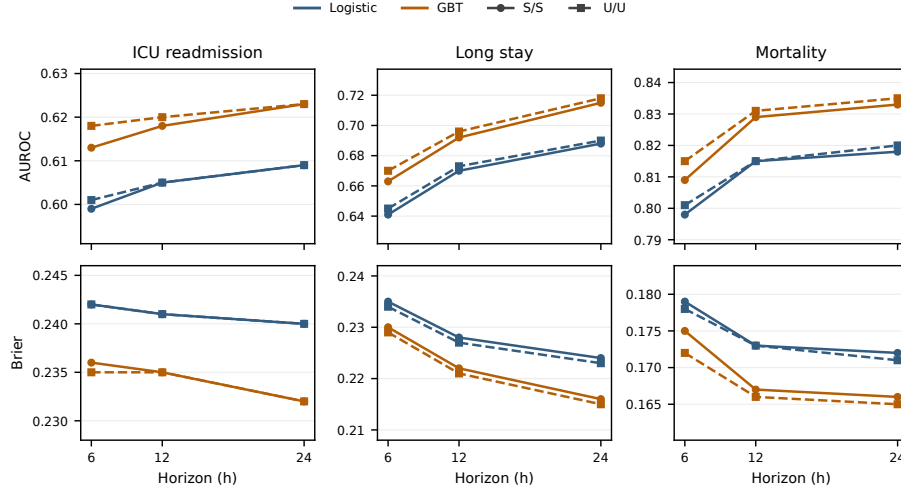


Fig. 5. Downstream prediction effects are model and task dependent rather than directionally fixed. Safe/safe and occurrence only unsafe/unsafe evaluations are shown separately for logistic regression and gradient boosted tree models across 6h, 12h, and 24h horizons.

assumptions explicit, auditable, and comparable. Visibility constraints have task dependent downstream effects: they may change evaluation results for some tasks, while for others they primarily improve the data semantics and audit boundary.

7 Limitations and Conclusion

ChronoGuard’s semantics depend on declared contracts and on the availability roles exposed by the source data. When an independent availability field is absent, strict point in time extraction must either exclude the table or use a low confidence proxy. Streaming replay is bounded and is used for correctness and cost characterization, not for a production real time claim. The downstream prediction study is an impact audit; it does not show that safe filtering universally improves accuracy or that occurrence only evaluation is always optimistic.

Within these limits, the evidence supports the central claim: longitudinal EHR analytics requires availability time as a first class data management semantic. By making timestamp roles executable, auditable, and reproducible, ChronoGuard-EHR prevents decision time queries from using facts that were not yet visible and exposes the cost of enforcing that semantics across batch and streaming workloads.

References

1. Alistair E. W. Johnson, Lucas Bulgarelli, Lu Shen, Alvin Gayles, Ayad Shammout, Steven Horng, Tom J. Pollard, Sicheng Hao, Benjamin Moody, Brian Gow, et al. MIMIC-IV, a freely accessible electronic health record dataset. *Scientific Data*, 10(1):1, 2023.
2. Tom J. Pollard, Alistair E. W. Johnson, Jesse D. Raffa, Leo A. Celi, Roger G. Mark, and Omar Badawi. The eICU collaborative research database, a freely available multi-center database for critical care research. *Scientific Data*, 5(1):180178, 2018.
3. Hrayr Harutyunyan, Hrant Khachatryan, David C. Kale, Greg Ver Steeg, and Aram Galstyan. Multitask learning and benchmarking with clinical time series data. *Scientific Data*, 6(1):96, 2019.
4. Sayash Kapoor and Arvind Narayanan. Leakage and the reproducibility crisis in machine-learning-based science. *Patterns*, 4(9), 2023.
5. Richard T. Snodgrass. *Developing Time-Oriented Database Applications in SQL*. Morgan Kaufmann Publishers Inc., 1999.
6. Curtis Dyreson, Fabio Grandi, Wolfgang Käfer, Nick Kline, Nikos Lorentzos, Yannis Mitsopoulos, Angelo Montanari, Daniel Nonen, Elisa Peressi, Barbara Pernici, et al. A consensus glossary of temporal database concepts. *ACM SIGMOD Record*, 23(1):52–64, 1994.
7. Neoklis Polyzotis, Sudip Roy, Steven Euijong Whang, and Martin Zinkevich. Data management challenges in production machine learning. In *Proceedings of the 2017 ACM International Conference on Management of Data*, pages 1723–1726, 2017.
8. David Sculley, Gary Holt, Daniel Golovin, Eugene Davydov, Todd Phillips, Dietmar Ebner, Vinay Chaudhary, Michael Young, Jean-Francois Crespo, and Dan Dennison. Hidden technical debt in machine learning systems. *Advances in Neural Information Processing Systems*, 28, 2015.
9. Sebastian Schelter, Dustin Lange, Philipp Schmidt, Meltem Celikel, and Felix Biessmann. Automating large-scale data quality verification. *arXiv preprint*, 2018.
10. Nicole Gray Weiskopf and Chunhua Weng. Methods and dimensions of electronic health record data quality assessment: enabling reuse for clinical research. *Journal of the American Medical Informatics Association*, 20(1):144–151, 2013.
11. Michael G. Kahn, Tiffany J. Callahan, Juliana Barnard, Alan E. Bauck, Jeff Brown, Bruce N. Davidson, Hossein Estiri, Carsten Goerg, Erin Holve, Steven G. Johnson, et al. A harmonized data quality assessment terminology and framework for the secondary use of electronic health record data. *eGEMs*, 4(1):1244, 2016.
12. Jawad Chishtie, Natalie Sapiro, Natalie Wiebe, Leora Rabatach, Diane Lorenzetti, Alexander A. Leung, Doreen Rabi, Hude Quan, and Cathy A. Eastwood. Use of epic electronic health record system for health care research: scoping review. *Journal of Medical Internet Research*, 25:e51003, 2023.
13. Peter Buneman, Sanjeev Khanna, and Wang-Chiew Tan. Why and where: A characterization of data provenance. In *International Conference on Database Theory*, pages 316–330. Springer, 2001.
14. James Cheney, Laura Chiticariu, and Wang-Chiew Tan. Provenance in databases: Why, how, and where. *Foundations and Trends in Databases*, 1(4):379–474, 2009.
15. Alvin Rajkomar, Eyal Oren, Kai Chen, Andrew M. Dai, Nissan Hajaaj, Michaela Hardt, Peter J. Liu, Xiaobing Liu, Jake Marcus, Mimi Sun, et al. Scalable and accurate deep learning with electronic health records. *NPJ Digital Medicine*, 1(1):18, 2018.

16. Jiantong Du, Yan Li, Yawen Li, Liwen Liao, Zhihao Zhao, and Guanhua Ye. MedFuse: a multi-source data fusion framework for diabetic retinopathy lesion segmentation. *Frontiers of Computer Science*, 20(3):2003625, 2026.
17. Guanhua Ye, Hongzhi Yin, Tong Chen, Hongxu Chen, Lizhen Cui, and Xiangliang Zhang. FENet: A frequency extraction network for obstructive sleep apnea detection. *IEEE Journal of Biomedical and Health Informatics*, 25(8):2848–2856, 2021.
18. Jianwei Wang, Ying Zhang, Kai Wang, Xuemin Lin, and Wenjie Zhang. Missing data imputation with uncertainty-driven network. *Proceedings of the ACM on Management of Data*, 2(3):1–25, 2024.
19. Jiasheng Duan, Peng-Fei Zhang, Ruihong Qiu, and Zi Huang. Long short-term enhanced memory for sequential recommendation. *World Wide Web*, 26(2):561–583, 2023.
20. Guanhua Ye, Jifeng He, Yan Li, Junping Du, Zhe Xue, Yingxia Shao, Meiyu Liang, and Yawen Li. Rethink representation learning for Questionnaire Data. In *Proceedings of the AAAI Conference on Artificial Intelligence, Vol. 40, No. 33*, pages 27782–27790, 2026.
21. Yawen Li, Mengyu Zhuang, Guanhua Ye, Yan Li, Junheng Wang, Jinyi Zhou, and Pengfei Zhang. FeBT: A feature balancing transformer for corporate ESG forecasting. *IEEE Transactions on Knowledge and Data Engineering*, 2025.
22. Guanhua Ye, Tong Chen, Quoc Viet Hung Nguyen, and Hongzhi Yin. Heterogeneous decentralised machine unlearning with seed model distillation. *CAAI Transactions on Intelligence Technology*, 9(3):608–619, 2024.
23. Tyler Akidau, Robert Bradshaw, Craig Chambers, Slava Chernyak, Rafael J. Fernández-Moctezuma, Reuven Lax, Sam McVeety, Daniel Mills, Frances Perry, Eric Schmidt, et al. The dataflow model: A practical approach to balancing correctness, latency, and cost in massive-scale, unbounded, out-of-order data processing. *Proceedings of the VLDB Endowment*, 8(12):1792–1803, 2015.
24. Tyler Akidau, Edmon Begoli, Slava Chernyak, Fabian Hueske, Kathryn Knight, Kenneth Knowles, Daniel Mills, and Dan Sotolongo. Watermarks in stream processing systems: Semantics and comparative analysis of Apache Flink and Google Cloud Dataflow. Technical report, Oak Ridge National Laboratory (ORNL), Oak Ridge, TN, United States, 2021.
25. Paris Carbone, Asterios Katsifodimos, Stephan Ewen, Volker Markl, Seif Haridi, and Kostas Tzoumas. Apache Flink: Stream and batch processing in a single engine. *The Bulletin of the Technical Committee on Data Engineering*, 38(4), 2015.
26. Marios Fragkoulis, Paris Carbone, Vasiliki Kalavri, and Asterios Katsifodimos. A survey on the evolution of stream processing systems. *The VLDB Journal*, 33(2):507–541, 2024.